Name: Varshitha Darisa

Reg no:24011101024

Roll no:24110098

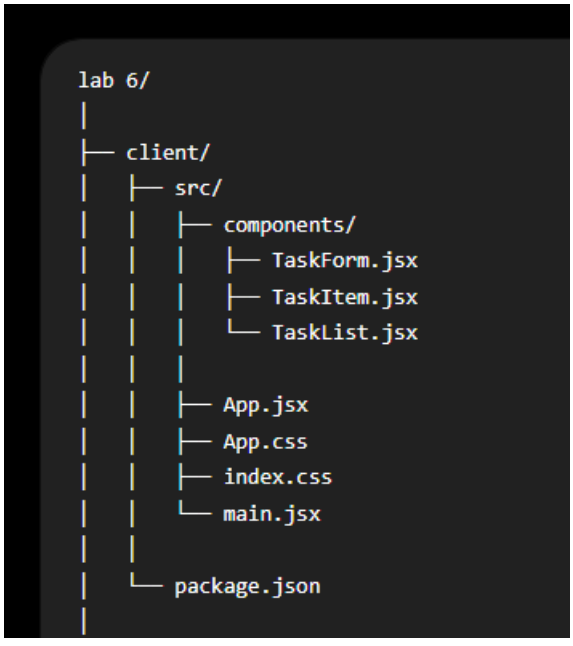
Class : AIDS-'A'

Web Technologies Lab 9

Aim: To develop and implement a Todo application using the MERN (MongoDB, Express.js, React.js, and Node.js) stack, with MongoDB Atlas for data storage and CRUD operations for managing tasks.

Code: Frontend

Folder structure for frontend



1. **App.jsx**

```
import { useEffect, useState } from "react";
import axios from "axios";
import "./App.css";

import TaskForm from "./components/TaskForm";
import TaskList from "./components/TaskList";

const API_URL = "http://localhost:5000/api/todos";
```

```
10.
11.function App() {
12.  const [todos, setTodos] = useState([]);
13.
14.  // Fetch todos when application loads
15.  useEffect(() => {
16.    fetchTodos();
17.  }, []);
18.
19.  const fetchTodos = async () => {
20.    try {
21.      const response = await axios.get(API_URL);
22.      setTodos(response.data);
23.    } catch (error) {
24.      console.error("Error fetching todos:", error);
25.    }
26.  };
27.
28.  // Add todo
29.  const addTodo = async (task) => {
30.    try {
31.      const response = await axios.post(API_URL, {
32.        task: task,
33.      });
34.
35.      setTodos((prevTodos) => [
36.        response.data,
37.        ...prevTodos,
38.      ]);
39.    } catch (error) {
40.      console.error("Error adding todo:", error);
41.    }
42.  };
43.
44.  // Toggle completed status
45.  const toggleTodo = async (todo) => {
46.    try {
47.      const response = await axios.put(
48.        `${API_URL}/${todo._id}`,
49.        {
50.          task: todo.task,
51.          completed: !todo.completed,
52.        }
53.      );
54.
55.      setTodos((prevTodos) =>
56.        prevTodos.map((item) =>
57.          item._id === todo._id ? response.data : item
```

```
        )
      );
    } catch (error) {
      console.error("Error updating todo:", error);
    }
  };

  // Edit todo
  const editTodo = async (todo) => {
    const updatedTask = window.prompt(
      "Edit your task:",
      todo.task
    );

    if (updatedTask === null) {
      return;
    }

    if (!updatedTask.trim()) {
      return;
    }

    try {
      const response = await axios.put(
        `${API_URL}/${todo._id}`,
        {
          task: updatedTask.trim(),
          completed: todo.completed,
        }
      );

      setTodos((prevTodos) =>
        prevTodos.map((item) =>
          item._id === todo._id ? response.data : item
        )
      );
    } catch (error) {
      console.error("Error editing todo:", error);
    }
  };

  // Delete todo
        const deleteTodo = async (id) => {
          const confirmDelete = window.confirm(
            "Are you sure you want to delete this task?"
          );

          if (!confirmDelete) {
```

```
106.           return;
107.         }
108.
109.         try {
110.           await axios.delete(`${API_URL}/${id}`);
111.
112.           setTodos((prevTodos) =>
113.             prevTodos.filter((todo) => todo._id !== id)
114.           );
115.         } catch (error) {
116.           console.error("Error deleting todo:", error);
117.         }
118.       };
119.
120.       const totalTasks = todos.length;
121.
122.       const completedTasks = todos.filter(
123.         (todo) => todo.completed
124.       ).length;
125.
126.       const pendingTasks = totalTasks - completedTasks;
127.
128.       return (
129.         <div className="app">
130.           <div className="todo-container">
131.
132.             {/* Header */}
133.             <div className="header">
134.               <div>
135.                 <p className="subtitle">MY TASK MANAGER</p>
136.
137.                 <h1>
138.                   MERN <span>Todo</span>
139.                 </h1>
140.
141.                 <p className="description">
142.                   Organize your tasks and stay productive.
143.                 </p>
144.               </div>
145.             </div>
146.
147.             {/* Statistics */}
148.             <div className="stats">
149.
150.               <div className="stat-card">
151.                 <div className="stat-number">
152.                   {totalTasks}
153.                 </div>
```

```
                <div className="stat-label">
                  Total
                </div>
              </div>

              <div className="stat-card">
                <div className="stat-number pending">
                  {pendingTasks}
                </div>
                <div className="stat-label">
                  Pending
                </div>
              </div>

              <div className="stat-card">
                <div className="stat-number completed">
                  {completedTasks}
                </div>
                <div className="stat-label">
                  Completed
                </div>
              </div>

            </div>

            {/* Add Task */}
            <TaskForm onAdd={addTodo} />

            {/* Task List */}
            <TaskList
              todos={todos}
              onToggle={toggleTodo}
              onEdit={editTodo}
              onDelete={deleteTodo}
            />

          </div>
        </div>
      );
    }

    export default App;
```

2. App.css

```
* {
  box-sizing: border-box;
}
```

```css
body {
  margin: 0;
  font-family: "Segoe UI", Arial, sans-serif;
  background: #f4f7fb;
  color: #1f2937;
}

button,
input {
  font-family: inherit;
}

.app {
  min-height: 100vh;
  padding: 50px 20px;
}

.todo-container {
  width: 100%;
  max-width: 850px;
  margin: 0 auto;
  background: white;
  border-radius: 24px;
  padding: 40px;
  box-shadow: 0 15px 45px rgba(0, 0, 0, 0.08);
}

/* ---------------- HEADER ---------------- */

.header {
  margin-bottom: 30px;
}

.subtitle {
  margin: 0 0 8px;
  font-size: 13px;
  font-weight: 700;
  letter-spacing: 2px;
  color: #6b7280;
}

.header h1 {
  margin: 0;
  font-size: 44px;
  line-height: 1.15;
  font-weight: 800;
  color: #1f2937;
```

```css
}

.header h1 span {
  color: #6366f1;
}

.description {
  margin-top: 10px;
  margin-bottom: 0;
  color: #6b7280;
  font-size: 16px;
}

/* ---------------- STATISTICS ---------------- */

.stats {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: 15px;
  margin-bottom: 30px;
}

.stat-card {
  padding: 18px;
  border-radius: 15px;
  background: #f8fafc;
  border: 1px solid #e5e7eb;
  text-align: center;
}

.stat-number {
  font-size: 28px;
  font-weight: 800;
  color: #4f46e5;
}

.stat-number.pending {
  color: #f59e0b;
}

.stat-number.completed {
  color: #10b981;
}

.stat-label {
  margin-top: 4px;
  font-size: 13px;
  color: #6b7280;
```

```css
}

/* ---------------- FORM ---------------- */

.task-form {
  display: flex;
  gap: 12px;
  margin-bottom: 25px;
}

.task-form input {
  color:black;
  flex: 1;
  min-width: 0;
  padding: 15px 18px;
  font-size: 15px;
  border: 1px solid #d1d5db;
  border-radius: 12px;
  outline: none;
  background: #f9fafb;
  transition: 0.2s;
}

.task-form input:focus {
  background: white;
  border-color: #6366f1;
  box-shadow: 0 0 0 3px rgba(99, 102, 241, 0.12);
}

.task-form button {
  padding: 0 22px;
  border: none;
  border-radius: 12px;
  background: #6366f1;
  color: white;
  font-size: 15px;
  font-weight: 600;
  cursor: pointer;
  transition: 0.2s;
}

.task-form button:hover {
  background: #4f46e5;
  transform: translateY(-1px);
}

/* ---------------- TASK LIST ---------------- */
```

```css
.task-list {
  display: flex;
  flex-direction: column;
  gap: 12px;
}

/* ---------------- TASK ITEM ---------------- */

.task-item {
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: 15px;
  padding: 17px;
  border: 1px solid #e5e7eb;
  border-radius: 15px;
  background: white;
  transition: 0.2s;
}

.task-item:hover {
  border-color: #c7d2fe;
  box-shadow: 0 5px 18px rgba(0, 0, 0, 0.05);
  transform: translateY(-1px);
}

.task-left {
  display: flex;
  align-items: center;
  gap: 14px;
  min-width: 0;
}

.check-button {
  width: 27px;
  height: 27px;
  flex-shrink: 0;
  border-radius: 50%;
  border: 2px solid #cbd5e1;
  background: white;
  color: white;
  font-size: 16px;
  font-weight: bold;
  cursor: pointer;
  display: flex;
  align-items: center;
  justify-content: center;
  transition: 0.2s;
```

```css
}

.check-button:hover {
  border-color: #6366f1;
}

.check-button.checked {
  border-color: #10b981;
  background: #10b981;
}

.task-content {
  display: flex;
  flex-direction: column;
  gap: 6px;
  min-width: 0;
}

.task-text {
  font-size: 16px;
  font-weight: 600;
  word-break: break-word;
}

.completed-task .task-text {
  color: #9ca3af;
  text-decoration: line-through;
}

.status {
  width: fit-content;
  padding: 3px 9px;
  border-radius: 20px;
  font-size: 11px;
  font-weight: 700;
}

.status-pending {
  color: #b45309;
  background: #fef3c7;
}

.status-completed {
  color: #047857;
  background: #d1fae5;
}

/* ---------------- BUTTONS ---------------- */
```

```css
.task-actions {
  display: flex;
  gap: 8px;
  flex-shrink: 0;
}

.edit-button,
.delete-button {
  border: none;
  padding: 8px 13px;
  border-radius: 8px;
  font-size: 13px;
  font-weight: 600;
  cursor: pointer;
  transition: 0.2s;
}

.edit-button {
  background: #eef2ff;
  color: #4f46e5;
}

.edit-button:hover {
  background: #e0e7ff;
}

.delete-button {
  background: #fef2f2;
  color: #dc2626;
}

.delete-button:hover {
  background: #fee2e2;
}

/* ---------------- EMPTY STATE ---------------- */

.empty-state {
  text-align: center;
  padding: 50px 20px;
  border: 2px dashed #e5e7eb;
  border-radius: 15px;
}

.empty-icon {
  width: 55px;
  height: 55px;
```

```
  margin: 0 auto 15px;
  display: flex;
  align-items: center;
  justify-content: center;
  border-radius: 50%;
  background: #eef2ff;
  color: #6366f1;
  font-size: 25px;
  font-weight: bold;
}

.empty-state h3 {
  margin: 0;
  font-size: 19px;
}

.empty-state p {
  margin-top: 8px;
  color: #6b7280;
}

/* ---------------- RESPONSIVE ---------------- */

@media (max-width: 650px) {
  .app {
    padding: 20px 12px;
  }

  .todo-container {
    padding: 25px 18px;
    border-radius: 18px;
  }

  .header h1 {
    font-size: 34px;
  }

  .stats {
    gap: 8px;
  }

  .stat-card {
    padding: 12px 5px;
  }

  .task-form {
    flex-direction: column;
  }
```

```
  .task-form button {
    padding: 13px;
  }

  .task-item {
    align-items: flex-start;
  }

  .task-actions {
    flex-direction: column;
  }
}
```

3. components/TaskForm.jsx

```
import { useState } from "react";

function TaskForm({ onAdd }) {
  const [newTodo, setNewTodo] = useState("");

  const handleSubmit = (e) => {
    e.preventDefault();

    if (!newTodo.trim()) {
      return;
    }

    onAdd(newTodo.trim());

    setNewTodo("");
  };

  return (
    <form className="task-form" onSubmit={handleSubmit}>

      <input
        type="text"
        value={newTodo}
        onChange={(e) =>
          setNewTodo(e.target.value)
        }
        placeholder="What do you need to accomplish?"
      />

      <button type="submit">
        + Add Task
      </button>
```

```
    </form>
  );
}

export default TaskForm;
```

4. components/TaskList.jsx

```
import TaskItem from "./TaskItem";

function TaskList({
  todos,
  onToggle,
  onEdit,
  onDelete,
}) {
  if (todos.length === 0) {
    return (
      <div className="empty-state">
        <div className="empty-icon">✓</div>

        <h3>No tasks yet</h3>

        <p>
          Add your first task and start getting things done.
        </p>
      </div>
    );
  }

  return (
    <div className="task-list">
      {todos.map((todo) => (
        <TaskItem
          key={todo._id}
          todo={todo}
          onToggle={onToggle}
          onEdit={onEdit}
          onDelete={onDelete}
        />
      ))}
    </div>
  );
}

export default TaskList;
```

4. components/TaskItem.jsx

```
function TaskItem({
  todo,
  onToggle,
  onEdit,
  onDelete,
}) {
  return (
    <div
      className={`task-item ${
        todo.completed ? "completed-task" : ""
      }`}
    >

      <div className="task-left">

        <button
          className={`check-button ${
            todo.completed ? "checked" : ""
          }`}
          onClick={() => onToggle(todo)}
          title={
            todo.completed
              ? "Mark as pending"
              : "Mark as completed"
          }
        >
          {todo.completed ? "✓" : ""}
        </button>

        <div className="task-content">

          <span className="task-text">
            {todo.task}
          </span>

          <span
            className={`status ${
              todo.completed
                ? "status-completed"
                : "status-pending"
            }`}
          >
            {todo.completed
              ? "Completed"
              : "Pending"}
          </span>
```

```
        </div>

      </div>

      <div className="task-actions">

        <button
          className="edit-button"
          onClick={() => onEdit(todo)}
          title="Edit task"
        >
          Edit
        </button>

        <button
          className="delete-button"
          onClick={() => onDelete(todo._id)}
          title="Delete task"
        >
          Delete
        </button>

      </div>

    </div>
  );
}


export default TaskItem;
```

Code:Backend

Folder structure

```
└── server/
    ├── models/
    │   └── Todo.js
    │
    ├── routes/
    │   └── todoRoutes.js
    │
    ├── .env
    ├── server.js
    └── package.json
```

1. **models/Todo.js**

```
const mongoose = require("mongoose");

const todoSchema = new mongoose.Schema(
  {
    task: {
      type: String,
      required: true,
      trim: true,
    },

    completed: {
      type: Boolean,
      default: false,
    },
  },
  {
    timestamps: true,
  }
);

module.exports = mongoose.model("Todo", todoSchema);
```

2. routes/todoRoutes.js

```
const express = require("express");
const Todo = require("../models/Todo");

const router = express.Router();

// GET all todos
router.get("/", async (req, res) => {
  try {
    const todos = await Todo.find().sort({ createdAt: -1 });
    res.json(todos);
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
});

// POST - create a todo
router.post("/", async (req, res) => {
  try {
    if (!req.body.task || !req.body.task.trim()) {
      return res.status(400).json({
        message: "Task cannot be empty",
      });
    }

    const todo = new Todo({
```

```
      task: req.body.task.trim(),
      completed: false,
    });

    const savedTodo = await todo.save();

    res.status(201).json(savedTodo);
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
});

// PUT - update a todo
router.put("/:id", async (req, res) => {
  try {
    const updatedTodo = await Todo.findByIdAndUpdate(
      req.params.id,
      {
        task: req.body.task,
        completed: req.body.completed,
      },
      {
        new: true,
        runValidators: true,
      }
    );

    if (!updatedTodo) {
      return res.status(404).json({
        message: "Todo not found",
      });
    }

    res.json(updatedTodo);
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
});

// DELETE - delete a todo
router.delete("/:id", async (req, res) => {
  try {
    const deletedTodo = await Todo.findByIdAndDelete(req.params.id);

    if (!deletedTodo) {
      return res.status(404).json({
        message: "Todo not found",
      });
```

```
    }

    res.json({
      message: "Todo deleted successfully",
      id: req.params.id,
    });
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
});

module.exports = router;
```

3. Server.js

```
const express = require("express");
const mongoose = require("mongoose");
const cors = require("cors");
require("dotenv").config();

const todoRoutes = require("./routes/todoRoutes");

const app = express();

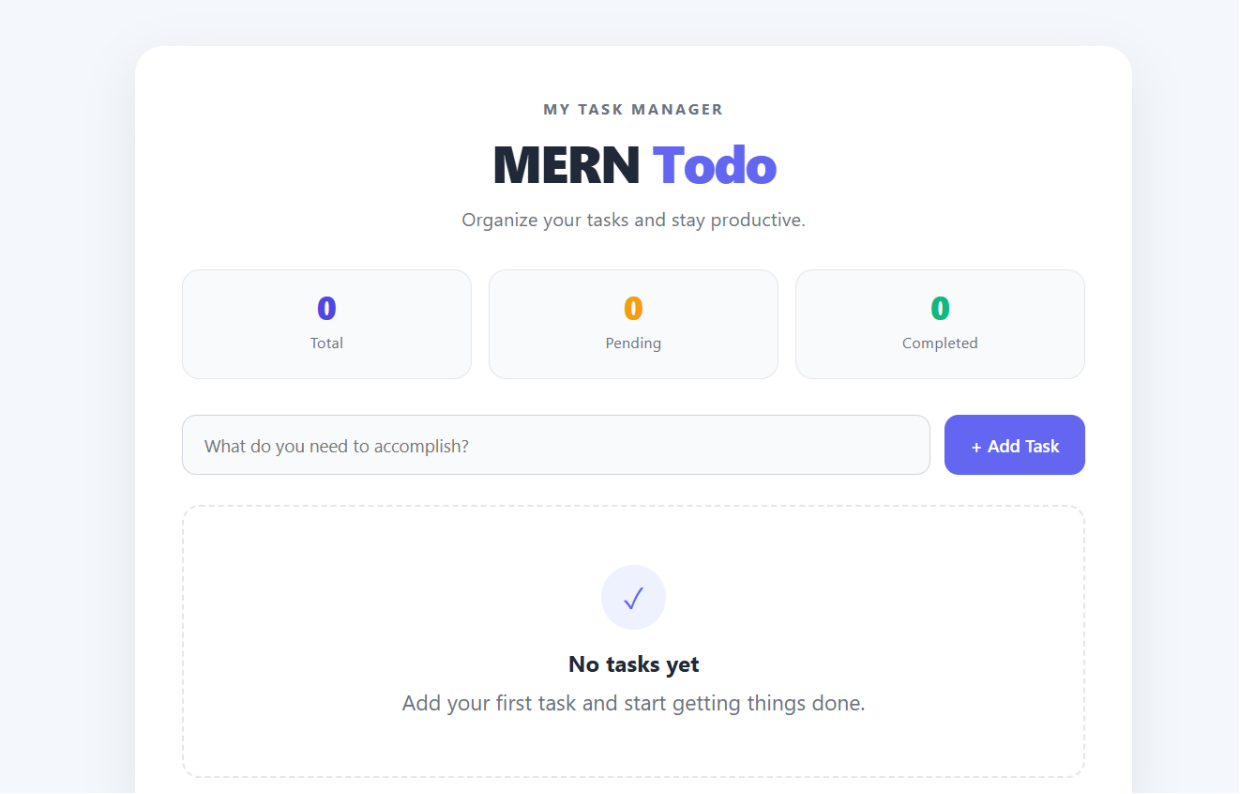
app.use(cors());
app.use(express.json());

app.use("/api/todos", todoRoutes);

mongoose
  .connect(process.env.MONGO_URI)
  .then(() => {
    console.log("MongoDB connected successfully");

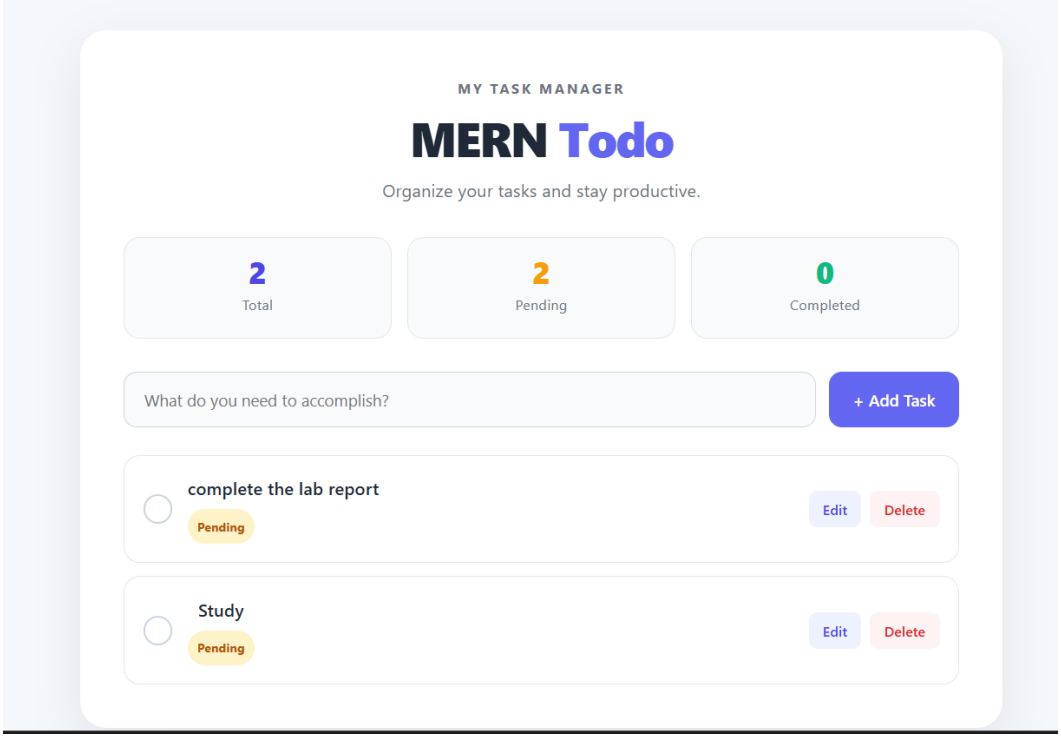
    app.listen(process.env.PORT, () => {
      console.log(`Server running on port ${process.env.PORT}`);
    });
  })
  .catch((error) => {
    console.error("MongoDB connection failed:", error);
  });
```

Screenshot of the website:

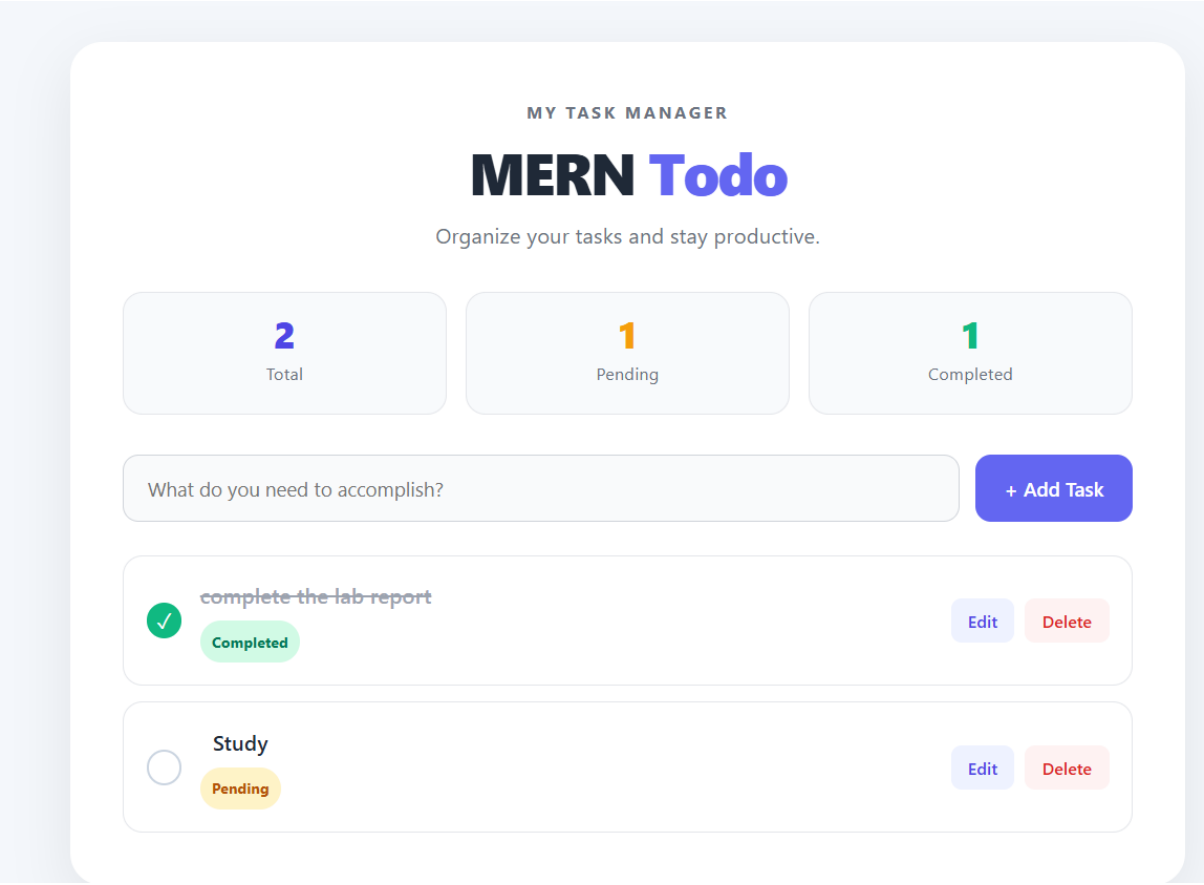


So far no task is added (displays empty)

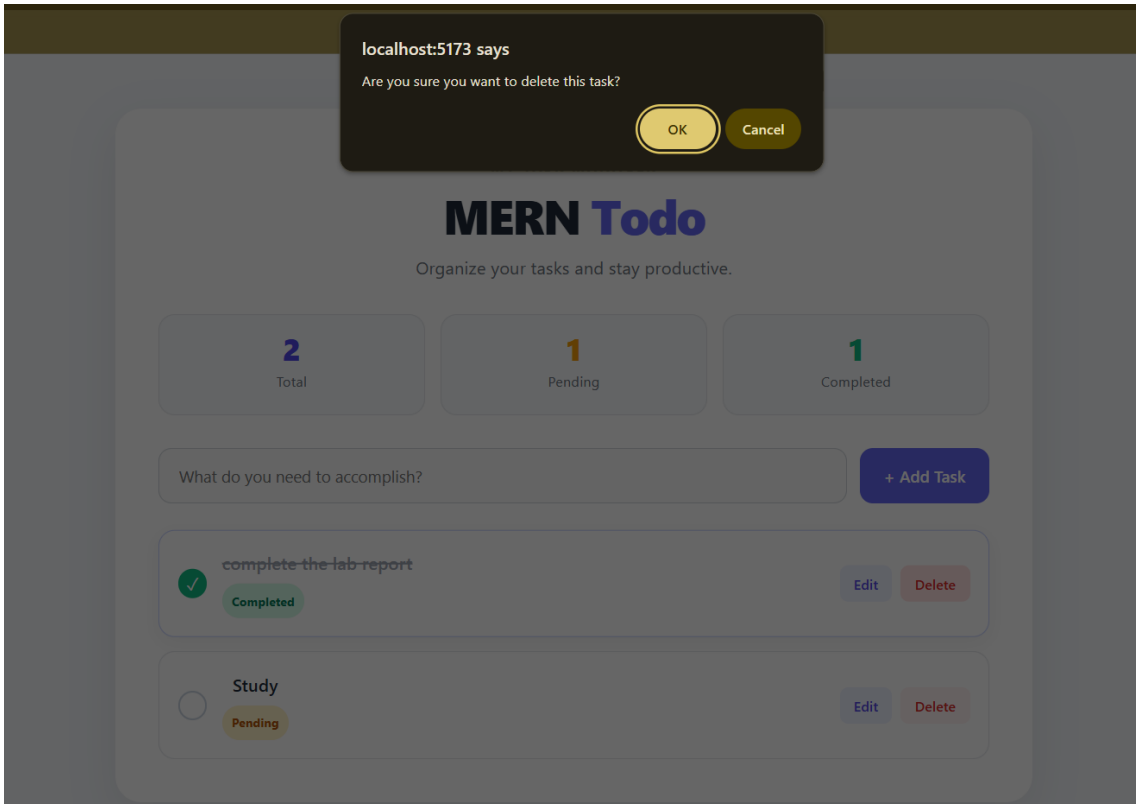
1. **Task is added(eg :**study, complete the lab report)
   Here post method is used to add the todo item



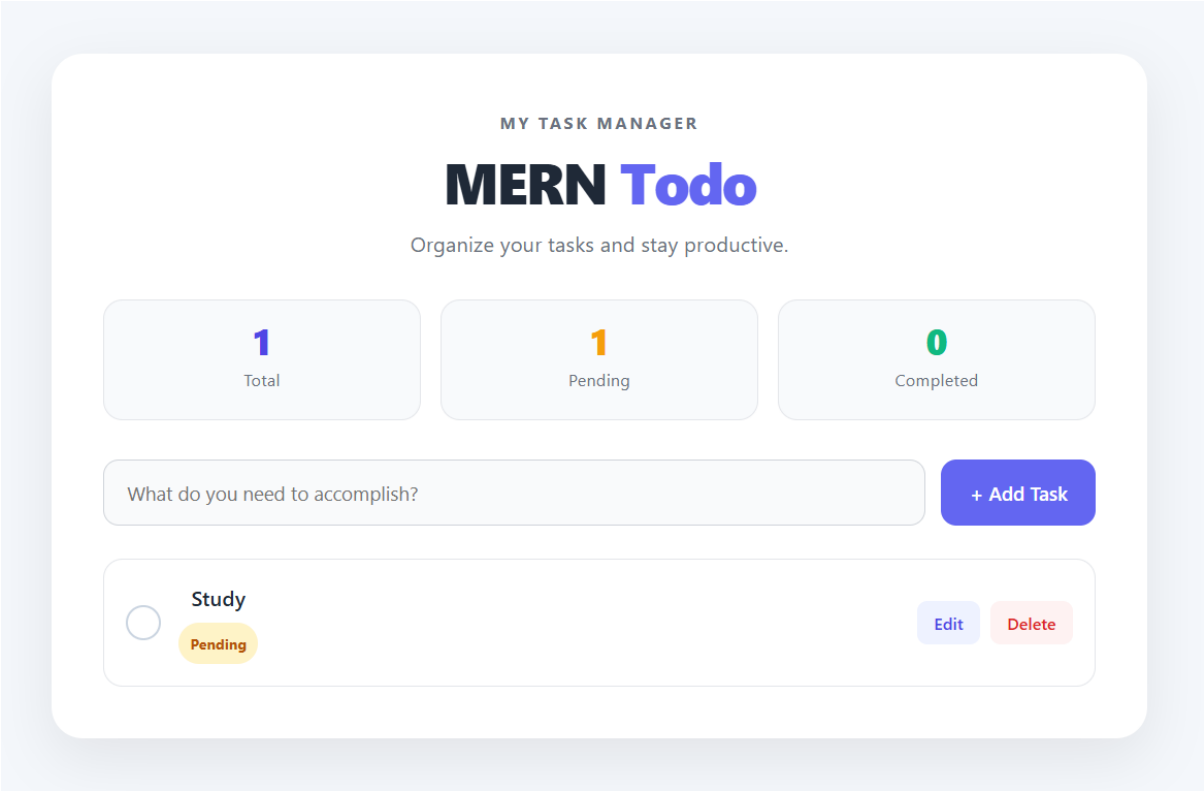
2. Put method to update the status of the task



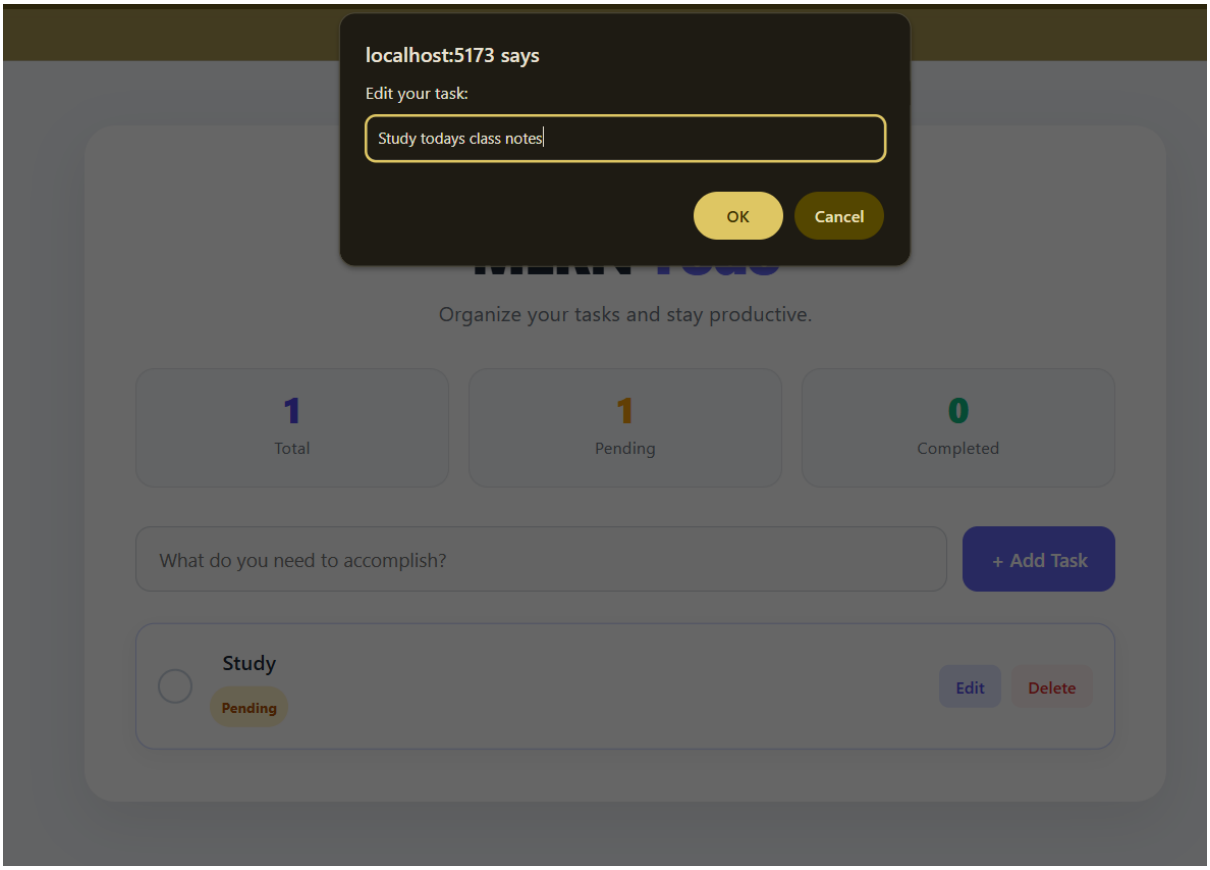
3. Delete method to delete the item

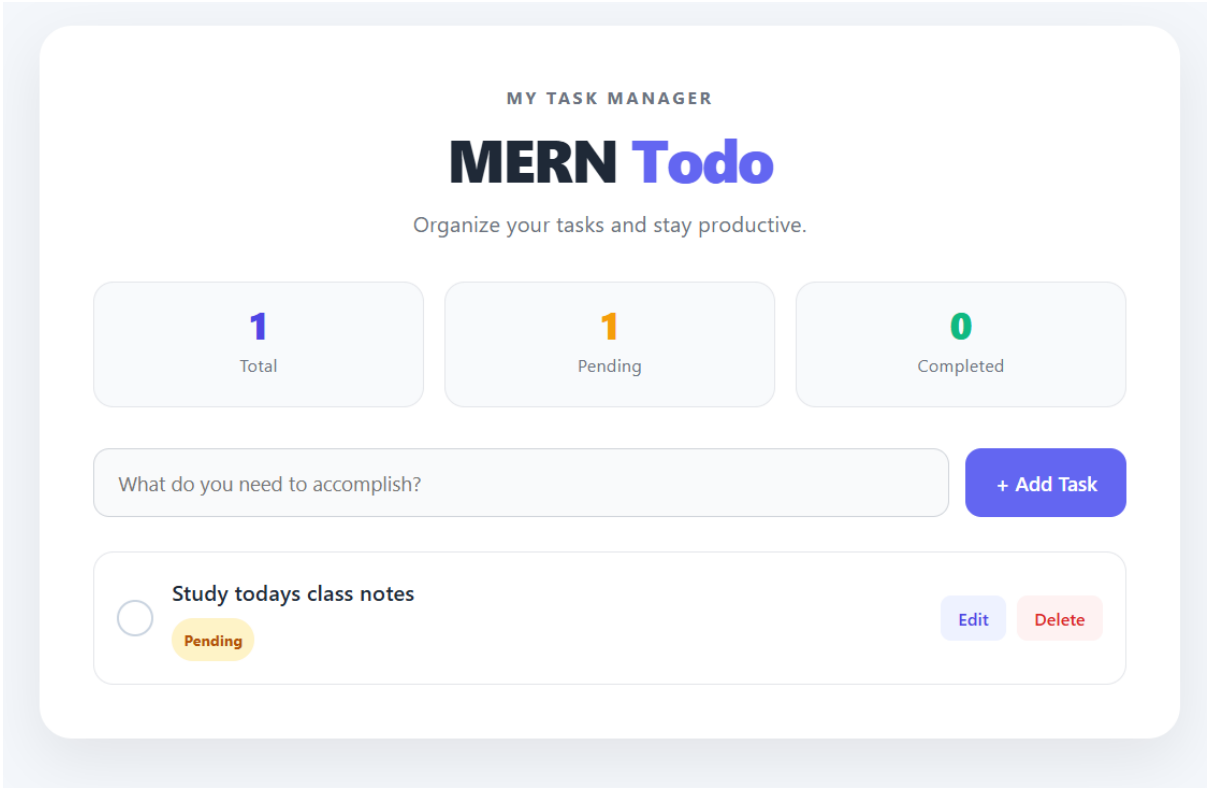


After deleting the completed task from the list



4. Editing the added task




MY TASK MANAGER
MERN Todo
Organize your tasks and stay productive.
1
Total
1
Pending
0
Completed
What do you need to accomplish?
+ Add Task
Study todays class notes
Pending
Edit
Delete